

CS 162

Computer Architecture

Midterm Review

University of California, Riverside

Bingyao Li
<http://libingyao.github.io/>

Material on exam



- **Slides 0 to 7 inclusive**
- Understand the performance metrics
- Understand pipeline workflow
- Understand hazards and solutions
- Understand dynamic parallelism

Computing CPU time

$$\text{Average Cycles per Instruction (CPI)} = \sum_{j=1}^n CPI_j * F_j$$

Where CPI_j is the number of cycles needed to execute instructions of type j , and F_j is the percentage (fraction) of instructions that are of type j .

Example: Base Machine (Reg / Reg)

Op	Freq	Cycles	$CPI_j * F_j$	(% Time)
ALU	50%	1	.5	(33%)
Load	20%	2	.4	(27%)
Store	10%	2	.2	(13%)
Branch	20%	2	.4	(27%)
			1.5	

Typical Mix

A Simple Performance Tradeoff Example



Op	Freq	CPI _i	Freq x CPI _i
ALU	50%	1	.5
Load	20%	5	1.0
Store	10%	3	.3
Branch	20%	2	.4
Average (Effective) CPI			$\Sigma = 2.2$

.5 .5 .25

.4 1.0 1.0

.3 .3 .3

.4 .2 .4

1.6 2.0 1.95

- How much faster would the machine be if a better data cache reduced the average load time to 2 cycles?

CPU time new = 1.6 x IC x CC so 2.2/1.6 means 37.5% faster

- How does this compare with using branch prediction to shave a cycle off the branch time?

CPU time new = 2.0 x IC x CC so 2.2/2.0 means 10% faster

- What if two ALU instructions could be executed at once?

CPU time new = 1.95 x IC x CC so 2.2/1.95 means 12.8% faster

$$CPU\ time = \text{total \# of instructions} \times CPI \times \text{Cycle time}$$

Example: For some programs,

Machine A has a clock cycle time of 10 ns. and a CPI of 2.0

Machine B has a clock cycle time of 20 ns. and a CPI of 1.2

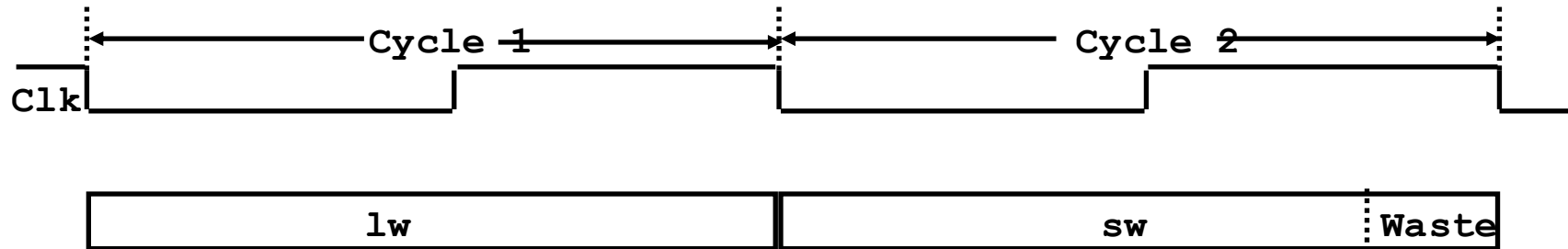
What machine is faster for this program?

$$CPI \times \text{Cycle time}$$

Machine A: cycle time = 10 ns, CPI = 2.0 Time per instruction = $2.0 \times 10\text{ ns} = 20\text{ ns}$.

Machine B: cycle time = 20 ns, CPI = 1.2 Time per instruction is $1.2 \times 20\text{ ns} = 24\text{ ns}$.

Single Cycle CPU



- Every instruction completes in exactly one cycle time
CPI = 1
- Uses the clock cycle inefficiently – the clock cycle must be timed to accommodate the **slowest** instruction
 - Load, needs address calculation

CCT Problem



- If all instructions must complete within one clock cycle, then the cycle time has to be large enough to accommodate the *slowest* instruction.
- For example, `lw $t0, -4($sp)` needs 8ns, assuming the delays shown here.

reading the instruction memory	2ns	}	8ns
reading the base register \$sp	1ns		
computing memory address \$sp-4	2ns		
reading the data memory	2ns		
storing data back to \$t0	1ns		

- In a pipeline, CCT is determined by the slowest single stage.

Limits to pipelining (Sections C.1 and C.2)



- **Hazards** prevent next instruction from executing during its designated clock cycle
 - **Structural hazards**: HW cannot support some combination of instructions.
 - **Data hazards**: An instruction depends on result of prior instruction still in the pipeline.
 - RAW -> forwarding
 - WAR, WAW -> register renaming
 - **Control hazards**: Pipelining of branches & other instructions *stall* the pipeline until the hazard *bubbles* in the pipeline
 - branch prediction

Example of Data Forwarding

Assume 4-stage, pipelined, FP add; 7-stage, pipelined, FP multiply; 1 stage, load/store. Further assume there are two register read ports, one write port, and separate I\$ and D\$ for instruction and data memory accesses.

1 fld F4, 0(R2)

2 fld F5, 8(R2)

3 fadd.d F6, F4, F5

4 fadd.d F8, F5, F6

5 fmult, F7, F6, F4

6 fsd F7 24(R2)

7 fsd F8 16(R2)

Instruction 1	Instruction 2	register	
1	3	F4	RAW
1	5	F4	RAW
2	3	F5	RAW
2	4	F5	RAW
3	4	F6	RAW
3	5	F6	RAW
4	7	F8	RAW
5	6	F7	RAW

Example of Data Forwarding

Assume 4-stage, pipelined, FP add; 7-stage, pipelined, FP multiply; 1 stage, load/store. Further assume there are two register read ports, one write port, and separate I\$ and D\$ for instruction and data memory accesses.

Instruction	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
fld F4, 0(R2)	IF	ID	EX	MEM	WB															
fld F5, 8(R2)		IF	ID	EX	MEM	WB														
fadd.d F6, F4, F5			IF	ID	ID	A1	A2	A3	A4	WB										
fadd.d F8, F5, F6				IF	ID	ID	ID	ID	ID	A1	A2	A3	A4	WB						
fmult, F7, F6, F4					IF	IF	IF	IF	IF	ID	M1	M2	M3	M4	M5	M6	M7	WB		
fsd F7 24(R2)										IF	ID	EX	EX	EX	EX	EX	EX	MEM	WB	
fsd F8 16(R2)											IF	ID	ID	ID	ID	ID	ID	EX	MEM	WB

Example of Data Forwarding

Assume 4-stage, pipelined, FP add; 7-stage, pipelined, FP multiply; 1 stage, load/store. Further assume there are two register read ports, one write port, and separate I\$ and D\$ for instruction and data memory accesses.

Instruction	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
fld F4, 0(R2)	IF	ID	EX	MEM	WB															
fld F5, 8(R2)		IF	ID	EX	MEM	WB														
fadd.d F6, F4, F5			IF	ID	ID	A1	A2	A3	A4	WB										
fadd.d F8, F5, F6				IF	IF	ID	ID	ID	ID	A1	A2	A3	A4	WB						
fmult, F7, F6, F4						IF	IF	IF	IF	ID	M1	M2	M3	M4	M5	M6	M7	WB		
fsd F7 24(R2)										IF	ID	EX	EX	EX	EX	EX	EX	MEM	WB	
fsd F8 16(R2)											IF	ID	ID	ID	ID	ID	ID	EX	MEM	WB

With forwarding: $0 + 1 + 3 + 0 + 5 + 0$

Renaming Example: Initial State



```
lw    $t0, 0($s1)
addu  $t0, $t0, $s2
sw    $t0, 0($s1)
sub   $t0, $s1, $s2
addi  $s1, $s1, -4
bne   $s1, $0, lp
```

\$s1	p1
\$s2	p2
\$t0	p3

Map Table

p4
p5
p6
p7
p8

Free List

Renaming Example: $\perp_w$ Renaming



Over-written Reg

```
lw    $t0, 0($s1) →  
addu  $t0, $t0, $s2  
sw    $t0, 0($s1)  
sub   $t0, $s1, $s2  
addi  $s1, $s1, -4  
bne   $s1, $0, lp
```

\$s1	p1
\$s2	p2
\$t0	p3

Map Table

p4
p5
p6
p7
p8

Free List

Renaming Example: lw Renaming



Over-written Reg

```
lw    $t0, 0($s1)  →  lw    p4, 0(p1)
addu  $t0, $t0, $s2
sw    $t0, 0($s1)
sub   $t0, $s1, $s2
addi  $s1, $s1, -4
bne   $s1, $0, lp
```

\$s1	p1
\$s2	p2
\$t0	p3

Map Table

p4
p5
p6
p7
p8

Free List

Renaming Example: lw Renaming



Over-written Reg

```
lw    $t0, 0($s1)  →  lw    p4, 0(p1)
addu  $t0, $t0, $s2
sw    $t0, 0($s1)
sub   $t0, $s1, $s2
addi  $s1, $s1, -4
bne   $s1, $0, lp
```

\$s1	p1
\$s2	p2
\$t0	p4

Map Table

p5
p6
p7
p8

Free List

Renaming Example: lw Renaming



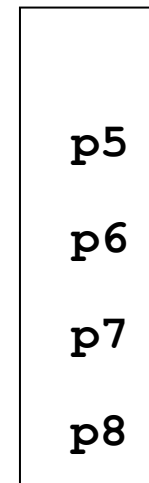
Over-written Reg

`lw $t0, 0($s1) → lw p4, 0(p1) [p3]`
`addu $t0, $t0, $s2`
`sw $t0, 0($s1)`
`sub $t0, $s1, $s2`
`addi $s1, $s1, -4`
`bne $s1, $0, lp`

\$s1	p1
\$s2	p2
\$t0	p4

p3

Map Table



Free List

Renaming Example: addu Renaming



Over-written Reg

```

lw    $t0, 0($s1)      lw    p4, 0(p1)      [p3]
addu  $t0, $t0, $s2 →  addu  p5, p4, p2
sw    $t0, 0($s1)
sub   $t0, $s1, $s2
addi  $s1, $s1, -4
bne   $s1, $0, lp
    
```

\$s1	p1
\$s2	p2
\$t0	p4

p3

Map Table

p5
p6
p7
p8

Free List

Renaming Example: addu Renaming



Over-written Reg

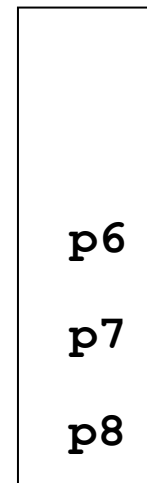
```

lw    $t0, 0($s1)      lw    p4, 0(p1)      [p3]
addu  $t0, $t0, $s2 →  addu  p5, p4, p2      [p4]
sw    $t0, 0($s1)
sub   $t0, $s1, $s2
addi  $s1, $s1, -4
bne   $s1, $0, lp
    
```

\$s1	p1
\$s2	p2
\$t0	p5

p4 p3

Map Table



Free List

Renaming Example: *sw* Renaming



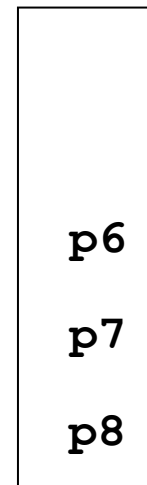
Over-written Reg

lw	\$t0, 0(\$s1)		lw	p4, 0(p1)	[p3]
addu	\$t0, \$t0, \$s2		addu	p5, p4, p2	[p4]
sw	\$t0, 0(\$s1)	→	sw	p5, 0(p1)	
sub	\$t0, \$s1, \$s2				
addi	\$s1, \$s1, -4				
bne	\$s1, \$0, lp				

\$s1	p1
\$s2	p2
\$t0	p5

p4 p3

Map Table



Free List

Renaming Example: sub Renaming



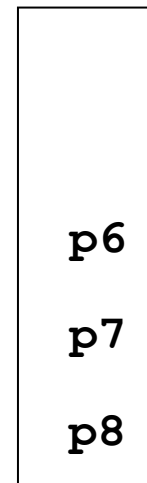
Over-written Reg

lw	\$t0, 0(\$s1)	lw	p4, 0(p1)	[p3]
addu	\$t0, \$t0, \$s2	addu	p5, p4, p2	[p4]
sw	\$t0, 0(\$s1)	sw	p5, 0(p1)	
sub	\$t0, \$s1, \$s2 →	sub	p6, p1, p2	
addi	\$s1, \$s1, -4			
bne	\$s1, \$0, lp			

\$s1	p1
\$s2	p2
\$t0	p5

p4 p3

Map Table



Free List

Renaming Example: sub Renaming



Over-written Reg

lw	\$t0, 0(\$s1)	lw	p4, 0(p1)	[p3]
addu	\$t0, \$t0, \$s2	addu	p5, p4, p2	[p4]
sw	\$t0, 0(\$s1)	sw	p5, 0(p1)	
sub	\$t0, \$s1, \$s2 →	sub	p6, p1, p2	[p5]
addi	\$s1, \$s1, -4			
bne	\$s1, \$0, lp			

\$s1	p1
\$s2	p2
\$t0	p6

p5 p4 p3



Map Table

Free List

Renaming Example: addi Renaming

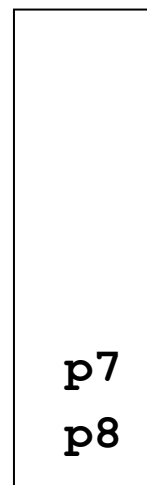


Over-written Reg

lw	\$t0, 0(\$s1)	lw	p4, 0(p1)	[p3]
addu	\$t0, \$t0, \$s2	addu	p5, p4, p2	[p4]
sw	\$t0, 0(\$s1)	sw	p5, 0(p1)	
sub	\$t0, \$s1, \$s2	sub	p6, p1, p2	[p5]
addi	\$s1, \$s1, -4 →	addi	p7, p1, -4	[p1]
bne	\$s1, \$0, lp			

\$s1	p1
\$s2	p2
\$t0	p6

p5 p4 p3



Map Table

Free List

Renaming Example: addi Renaming

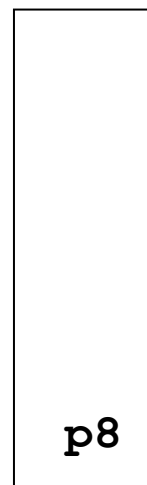


Over-written Reg

lw	\$t0, 0(\$s1)	lw	p4, 0(p1)	[p3]
addu	\$t0, \$t0, \$s2	addu	p5, p4, p2	[p4]
sw	\$t0, 0(\$s1)	sw	p5, 0(p1)	
sub	\$t0, \$s1, \$s2	sub	p6, p1, p2	[p5]
addi	\$s1, \$s1, -4 →	addi	p7, p1, -4	[p1]
bne	\$s1, \$0, lp			

\$s1	p7	p1
\$s2	p2	
\$t0	p6	p5 p4 p3

Map Table



Free List

Renaming Example: bne Renaming

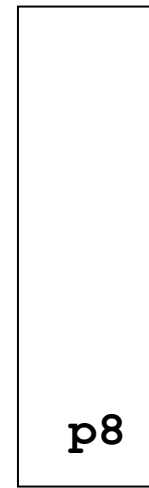


Over-written Reg

lw	\$t0, 0(\$s1)	lw	p4, 0(p1)	[p3]
addu	\$t0, \$t0, \$s2	addu	p5, p4, p2	[p4]
sw	\$t0, 0(\$s1)	sw	p5, 0(p1)	
sub	\$t0, \$s1, \$s2	sub	p6, p1, p2	[p5]
addi	\$s1, \$s1, -4	addi	p7, p1, -4	[p1]
bne	\$s1, \$0, lp	→	bne	p7, p0, lp

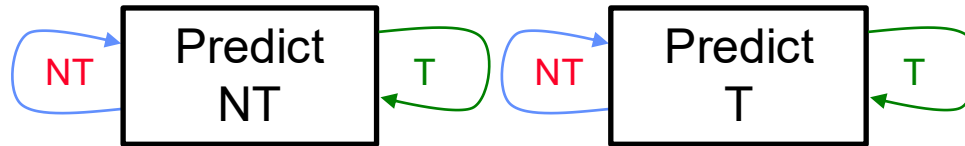
\$s1	p7	p1
\$s2	p2	
\$t0	p6	p5 p4 p3

Map Table



Free List

0-Bit Predictors, Loop for 10 Iterations

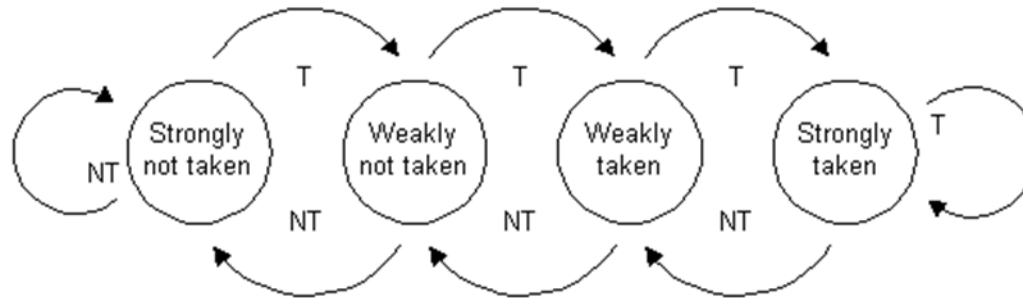


iteration	predict	actual	predict	actual
1	NT	T	T	T
2	NT	T	T	T
3	NT	T	T	T
4	NT	T	T	T
5	NT	T	T	T
6	NT	T	T	T
7	NT	T	T	T
8	NT	T	T	T
9	NT	T	T	T
10	NT	NT	T	NT
	10% accuracy		90% accuracy	

```

pre-loop instr
Loop:
  1st loop instr
  2nd loop instr
    .
    .
    .
  last loop instr
  bne $1,$2,Loop
post-loop instr
    
```

2-Bit Predictors



For the given sequence of branch outcomes: T, T, NT, NT, NT ... loop, start with SNT

Branch Prediction	Actual Branch behavior	Prediction Accuracy
S NT	T	✗
W NT	T	✗
W T	NT	✗
W NT	NT	✓
S NT	NT	✓

Branch Prediction	Actual Branch behavior	Prediction Accuracy
S NT	T	✗
W NT	T	✗
W T	NT	✗
W NT	NT	✓
S NT	NT	✓

The next iteration will look the **same** as the last state at the end of every iteration results in SNT.

The overall accuracy = $2/5 = 0.4 = 40\%$

The Tomasulo approach



- Main improvements over the scoreboard approach:
 - Uses forwarding on a Common Data Bus (CDB) -- more efficient dealing with RAW hazards,
 - avoids WAR hazards by reading the operands in the instruction-issue order, instead of stalling at issue. To accomplish this an instruction reads an available operand before waiting for the other.
 - Avoids WAW hazards by renaming the registers (using the *id* of a reservation station rather than the register *id*)
 - The control information and logic are distributed to the functional unit and not centralized.

Instruction		Issue	Execute	Write result
fld	F6, 34(R2)	X	X	
fld	F2, 45(R3)	X	X	
fmul.d	F0, F2, F4	X		
fsub.d	F8, F2, F6	X		
fdiv.d	F10, F0, F12	X		
fadd.d	F6, F8, F2	X		

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	Y	Load					34+Reg[R2]
Load2	Y	Load					45+Reg[R3]
Add1	Y	Sub			Load2	Load1	
Add2	Y	Add			Add1	Load2	
Add3	no						
Mult1	Y	Mul		Reg[F4]	Load2		
Mult2	Y	Div		Reg[F12]	Mult1		
Int	no						

	F0	F2	F4	F6	F8	F10	F12	...	F30
Qi	Mult1	load2		Add2	Add1	Mult2			

Instruction		Issue	Execute	Write result
fld	F6, 34(R2)	X	X	X
fld	F2, 45(R3)	X	X	
fmul.d	F0, F2, F4	X		
fsub.d	F8, F2, F6	X		
fdiv.d	F10, F0, F12	X		
fadd.d	F6, F8, F2	X		



Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	no						
Load2	Y	Load					45+Reg[R3]
Add1	Y	Sub			Load2		
Add2	Y	Add			Add1	Load2	
Add3	no						
Mult1	Y	Mul		Reg[F4]	Load2		
Mult2	Y	Div		Reg[F12]	Mult1		
Int	no						

	F0	F2	F4	F6	F8	F10	F12	...	F30
Qi	Mult1	load2		Add2	Add1	Mult2			

Instruction		Issue	Execute	Write result
fld	F6, 34(R2)	X	X	X
fld	F2, 45(R3)	X	X	X
fmul.d	F0, F2, F4	X		
fsub.d	F8, F2, F6	X		
fdiv.d	F10,F0,F12	X		
fadd.d	F6, F8, F2	X		



Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	no						
Load2	no						
Add1	Y	Sub	Reg[F2]	Reg[F6]	Load2		
Add2	Y	Add		Reg[F2]	Add1	Load2	
Add3	no						
Mult1	Y	Mul	Reg[F2]	Reg[F4]	Load2		
Mult2	Y	Div		Reg[F12]	Mult1		
Int	no						

	F0	F2	F4	F6	F8	F10	F12	...	F30
Qi	Mult1	Load2		Add2	Add1	Mult2			

Midterm Exam

Location: **WCH143**

Time: **Feb. 2 and Feb. 4 class time**

Duration: **55 mins**

No textbook, no cheat sheets, no calculator